

Introduction to Computers and Programming in C

Module V

File Handling

Files handling in C

File:

A file is a container in computer storage devices used for storing data.

Files handling in C

Why files are needed?

- **When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.**
- **If you have to enter a large number of data, it will take a lot of time to enter them all. However, if you have a file containing all the data, you can easily access the contents of the file using a few commands in C.**
- **You can easily move your data from one computer to another without any changes.**

Types of Files

There are two types of files you should know about:

- Text files
- Binary files

Files handling in C

Text files

- Text files are the normal **.txt** files. You can easily create text files using any simple text editors such as Notepad.
- When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.
- They take minimum effort to maintain, are easily readable, and provide the least security and takes bigger storage space.

Files handling in C

- **Binary files**
- Binary files are mostly the **.bin** files in your computer.
- Instead of storing data in plain text, they store it in the binary form (0's and 1's).
- They can hold a higher amount of data, are not readable easily, and provides better security than text files.

File operations

File Operations

In C, you can perform four major operations on files, either text or binary:

- Creating a new file
- Opening an existing file
- Closing a file
- Reading from and writing information to a file

Opening a File

- We need following information while defining a file in the secondary memory
- File Name
- Data Structure
- Purpose

Opening a File

- File Name: It is a string of characters that make up a valid file name for the operating system.
- It contains two name:
- Primary name
- Optional period with extension

For Example:

a.out

test.c

Opening a File

- Data Structure: Data Structure of a file is defined as FILE in the library of standard I/O function definitions. Therefore, all files should be declared as type FILE before they are used.
- File is a defined Data Type
- While opening a file we must specify the need such as writing the data to file or reading the data from the existing file.

Opening a file - for creation and edit

Opening a file is performed using the `fopen()` function defined in the `stdio.h` header file.

The syntax for opening a file in standard I/O is

```
FILE *fp;  
fp = fopen("filename", "mode");
```

For example,

```
fopen("E:\\cprogram\\newprogram.txt", "w");  
fopen("E:\\cprogram\\oldprogram.bin", "rb");
```

Opening Modes in Standard I/O

Mode	Meaning of Mode	During Inexistence of file
r	Open for reading.	If the file does not exist, fopen() returns NULL.
rb	Open for reading in binary mode.	If the file does not exist, fopen() returns NULL.
w	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a	Open for append. Data is added to the end of the file.	If the file does not exist, it will be created.
	Open for append in binary	If the file does not exist, it

Opening Modes in Standard I/O

ab	Open for append in binary mode. Data is added to the end of the file.	If the file does not exist, it will be created.
r+	Open for both reading and writing.	If the file does not exist, fopen() returns NULL.
rb+	Open for both reading and writing in binary mode.	If the file does not exist, fopen() returns NULL.

Opening Modes in Standard I/O

w+	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb+	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a+	Open for both reading and appending.	If the file does not exist, it will be created.

Opening Modes in Standard I/O

ab+	Open for both reading and appending in binary mode.	If the file does not exist, it will be created.
------------	--	--

Closing of the File

The file (both text and binary) should be closed after reading/writing operations.

Closing a file is performed using the `fclose()` function.

`fclose(file_pointer);`

Here `fp` is a file pointer associated with the file to be closed. Also files are automatically closed as soon as program terminates, however, it is good habit to close the file when ever required operations associated with file are complete.

Problems related to files

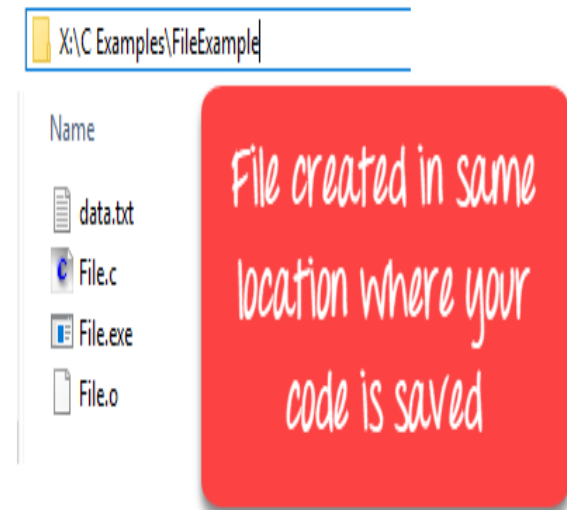
Opening a text file.

```
#include <stdio.h>

int main()
{
    FILE *fp;
    fp = fopen ("data.txt", "w");
}
```

Output:

File is created in the same folder where you have saved your code.



You can specify the path where you want to create your file

Writing to a File

- In C, when you write to a file, newline characters '\n' must be explicitly added.
- The stdio library offers the necessary functions to write to a file:
- **fputc(char, file_pointer):** It writes a character to the file pointed to by file_pointer.
- **fputs(str, file_pointer):** It writes a string to the file pointed to by file_pointer.
- **fprintf(file_pointer, str, variable_lists):** It prints a string to the file pointed to by file_pointer. The string can optionally include format specifiers and a list of variables variable_lists.

fputc() function

- The program below shows how to perform writing to a file:**fputc() Function:**

```
# include <stdio.h>
```

```
int main() {
```

```
    int i;
```

```
    FILE * fptr;
```

```
    char fn[50];
```

```
    char str[] = "Best of Luck \n";
```

```
    fptr = fopen("fputc_test.txt", "w"); // "w" defines "writing mode"
```

```
    for (i = 0; str[i] != '\n'; i++) {
```

```
        /* write to file using fputc() function */
```

```
        fputc(str[i], fptr);
```

```
    }
```

```
    fclose(fptr);
```

```
    return 0;
```

```
}
```

fputs() function

```
#include <stdio.h>

int main() {
    FILE * fp;
    fp = fopen("fputs_test.txt", "w+");
    fputs("Amity University ,", fp);
    fputs("Department of computer science\n", fp);
    fputs("Noida \n", fp);
    fclose(fp);
    return (0);
}
```

fprintf() function

```
#include <stdio.h>

int main() {
    FILE *fptr;
    fptr = fopen("fprintf_test.txt", "w"); // "w" defines "writing
mode"
    /* write to file */
    fprintf(fptr, "hello\n");
    fclose(fptr);
    return 0;
}
```

Reading Data from a File

There are three different functions dedicated to reading data from a file

- **fgetc(file_pointer):** It returns the next character from the file pointed to by the file pointer. When the end of the file has been reached, the EOF is sent back.
- **fgets(buffer, n, file_pointer):** It reads n-1 characters from the file and stores the string in a buffer in which the NULL character '\0' is appended as the last character.
- **fscanf(file_pointer, conversion_specifiers, variable_addresses):** It is used to parse and analyze data. It reads characters from the file and assigns the input to a list of variable pointers variable_addresses using conversion specifiers. Keep in mind that as with scanf, fscanf stops reading a string when space or newline is encountered.

Example

```
#include <stdio.h>

int main() {
    FILE * file_pointer;
    char buffer[30], c;
    file_pointer = fopen("fprintf_test.txt", "r");
    printf("----read a line----\n");
    fgets(buffer, 50, file_pointer);
    printf("%s\n", buffer);
    printf("----read and parse data----\n");
    file_pointer = fopen("fprintf_test.txt", "r"); //reset the pointer
    char str1[10], str2[2], str3[20], str4[2];
```

Example Contd...

```
fscanf(file_pointer, "%s %s %s %s", str1, str2, str3, str4);  
    printf("Read String1 |%s|\n", str1);  
    printf("Read String2 |%s|\n", str2);  
    printf("Read String3 |%s|\n", str3);  
    printf("Read String4 |%s|\n", str4);  
    printf("----read the entire file----\n");  
    file_pointer = fopen("fprintf_test.txt", "r"); //reset the  
pointer  
    while ((c = getc(file_pointer)) != EOF) printf("%c", c);  
    fclose(file_pointer);  
    return 0;  
}
```


getw() and putw()

- These are integer related functions.
- Their working are same as getc and putc function.
- These functions are used to read and write integer values respectively.

Syntax

```
putw(integer,fp);
```

```
getw(fp);
```

References

- Programming in ANSI C, E. Balagurusamy. McGrawHill
- Let Us C, Yashvant Kanetkar, BPB Publications
- Programming in C, Reema Thareja, Oxford University Press

Thank You